

CE903/913 System Requirements Specification and Project Plan

Authors: Yasmine Frizlen, Ana P. González, Zabed Ibne Emdad, Raghad Alsaqran, Gabriel Galván García, Myriam Raafat, Ricardo K. Flores, Pablo Del Campo Olano, Eduardo Dabbah Gindi

Supervisor: Dr. Muge Sayit

5th February 2026

Table of Contents

1	<i>Introduction</i>	4
1.1	Project aim and objectives	4
1.2	Acronyms	7
1.3	Overview	7
2	<i>Requirements Analysis and Specification</i>	9
2.1	Elicitation	9
2.2	Analysis	11
2.3	Specification	12
2.4	System Architecture	14
2.5	System Model	16
2.6	Functional Requirements	17
2.7	Non-functional requirements	21
	Product requirements	21
	Organizational requirements	22
	External requirements	23
3	<i>Verification, Validation and Testing</i>	24
3.1	Verification	24
3.2	Validation	24
3.3	Testing	24
3.3.1	Testing Approach	24
3.3.2	Testing Types	25
3.4	System Testing Schedule:	27
4	<i>Project Planning</i>	32
4.1	Project management structure and methodology	32
	Scrum	32
	Kanban	33
	Adopted Methodology: Scrumban	34
	Scrumban implementation	34
4.2	Project tools	37
	Jira	37
	Confluence	38

Miro.....	41
Kollabe.....	42
4.3 Project plan	43
4.4 Progress report.....	45
<i>References</i>	46

1 Introduction

This document constitutes the Requirements Specification for the development of an AI-enabled operational recommendation system for grid-connected photovoltaic (PV) and battery energy storage systems (BESS).

The purpose of this document is to define the project's scope, objectives, user requirements, and technical constraints. This establishes a shared understanding among all stakeholders, which includes the project team, academic supervisors, and our industry partner, *SNET Energy UK*.

This project is conducted in collaboration with *SNET Energy UK*, an AI consulting company operating in the renewable energy sector. Unlike typical academic group projects, this industry partnership situates the work within a real-world commercial context, requiring the team to balance academic requirements with practical applicability and stakeholder's expectations. The collaboration was established to explore AI-driven optimization techniques for BESS operational control, with findings intended to inform *SNET Energy UK*'s ongoing work in the energy domain.

The project is currently in its second sprint of a nine-sprint timeline. During the first two sprints, the team established core infrastructure (Jira for task management, Confluence for documentation, and GitLab for version control), defined recurring ceremonies (sprint reviews on Wednesdays, sprint planning on Thursdays, and weekly alignment meetings with on Mondays), and completed foundational research into the UK and Costa Rican energy markets, regulatory frameworks, and existing optimization approaches spanning both AI-based and traditional methods. The team has also decided to adopt an Agile methodology, specifically a Scrumban approach that merges Scrum's iterative sprints along with Kanban's workflow management principles.

This preparatory work has allowed the team to validate the project scope and proceed with well-informed requirements specification.

1.1 Project aim and objectives

Renewable energy systems have become more integrated into the electricity markets, adding focus on economically optimized control along traditional technical operations. Photovoltaic (PV) generation is variable, and when it is combined with a Battery Energy System Storage (BESS), a system operator must continuously make decisions on whether to store energy, sell the energy, or buy energy from the grid.

These decisions are primarily influenced by uncertain factors such as weather conditions, electricity market prices and the constraints of the battery systems. In order to handle this level of complexity, a smart, data-driven operation recommendation system is developed, which is the foundation of this project. This leads us to the project aim and its objectives:

- **Project aim:** This project aims to develop an AI-enabled operational recommendation system for a grid-connected PV-BESS installation operating in the UK intra-day continuous trading market.

This project aim is aligned with Dawson's definition of a project aim [1], this statement is intentionally broad, articulating the overall purpose of the project without prescribing measurable outcomes or operational detail.

The operational recommendation system (ORS) will be designed to provide profit-maximizing recommendations for battery charge and discharge actions, accounting for solar generation forecasts, real-time market prices, and physical system constraints. The work is conducted in collaboration with *SNET Energy UK*, who have provided domain expertise and guidance on commercial relevance. While the optimization framework is designed to be generalizable across different battery configurations and countries, evaluation will likely be conducted using a single-type BURST¹ battery in the UK intraday energy market.

Arising from the project's aim, four specific, measurable, appropriate, realistic, and time-bound (SMART) [2] objectives have been formulated to structure and assess the project's outcomes:

- **Objective 1: Baseline assessment and benchmarking**
To assess existing operational control strategies for grid-connected battery energy storage systems. This should include a representative simple static operational control strategy to establish a profit-based performance benchmark.
- **Objective 2: System formulation and parameterization**
To define the required input variables, system constraints, and modular system structure of the proposed system, including company- and country-specific parameters, without committing to a particular optimization or AI-based control strategy.

¹ A fast intra-day cycling configuration

- **Objective 3: AI-enabled control system**

To develop an AI-enabled operational recommendations system that determines feasible charging, discharging, buying, and selling actions for a grid-connected BESS, with the objective of maximizing profit subject to physical and market constraints.

- **Objective 4: Quantitative evaluation and comparison**

To quantitatively evaluate the performance of the proposed AI-enabled operational control system in the UK intraday market against the established static baseline model, using profit as the primary metric while ensuring compliance with operational feasibility constraints.

The operational requirements of the ORS have been captured as user stories, which describe system functionality from the perspective of an end user - in this case, a BESS operations operator. User stories provide a lightweight, stakeholder-focused means of articulating requirements that aligns well with the Agile methodology adopted for this project (see Section 4.1). These user stories subsequently inform the derivation of functional and non-functional requirements. Furthermore, user stories are mapped to sprint objectives and project milestones, ensuring traceability from stakeholder needs through to implemented and tested functionality.

The following user stories define the core functional requirements of the system:

- **US-1:** As a BESS operations operator, I want the system to retrieve relevant weather data from an external weather API, so that the system can estimate near-future solar generation to inform operational decisions.
- **US-2:** As a BESS operations operator, I want the system to use current and historical electricity market prices from my selected region, so that recommendations reflect real market conditions and recent price behavior.
- **US-3:** As a BESS operations operator, I want each decision cycle to obtain both weather/solar information and electricity price data before optimization runs, so that the recommendation is based on a complete and consistent set of inputs.
- **US-4:** As a BESS operations operator, I want the system to account for energy losses from power conversion and battery storage, so that the recommended strategy reflects realistically usable energy and profit.
- **US-5:** As a BESS operations operator, I want the system to compute a profit-maximizing operational recommendation using market prices, solar availability/current generation,

and battery constraints, so that I can operate the system for maximum expected revenue without violating system limits.

- **US-6:** As a BESS operations operator, I want the system to output one clear profit-maximizing action - charge the battery from solar, charge the battery from the grid, sell solar generation to the grid, or discharge the battery to sell to the grid - so that the decision can be executed unambiguously.

These user stories collectively address the end-to-end decision pipeline: data acquisition (US-1, US-2, US-3), realistic system modelling (US-4), optimization (US-5), and actionable output (US-6). The technical requirements and system architecture presented in subsequent sections are derived from these foundational user stories.

1.2 Acronyms

In Table 1, we define the acronyms used throughout the document.

Acronym	Definition
API	Application Programming Interface
BESS	Battery Energy Storage System
MVP	Minimum Viable Product
ORS	Operational Recommendation System
PV	Photovoltaic

Table 1: Table of acronyms used throughout document

1.3 Overview

The remainder of this document is structured as follows:

Section 2 presents the requirements analysis and specification, beginning with the elicitation methods employed to gather stakeholder needs, followed by the analysis process used to refine and prioritize these requirements. This section also details the system architecture and system model, illustrating the physical configuration of the PV-BESS installation and the interactions between system components. The functional requirements are then specified, derived directly from the user stories introduced in Section 1.1, alongside the non-functional requirements organized into product, organizational, and external categories.

Section 3 outlines the testing schedule, describing the unit, integration, and performance testing strategies that will be employed to validate system functionality, as well as the CI/CD automation approach adopted to support iterative development.

Finally, Section 4 addresses project planning, detailing the Scrumban methodology adopted by the team, the project management tools used to coordinate work, the project timeline as represented in the GANTT chart, and the progress achieved during initial sprints.

2 Requirements Analysis and Specification

This section starts by describing the methods that have been adopted for eliciting, analyzing, and specifying the requirements. It then proceeds to outline the architecture of the PV-battery system, its connection to the grid, and the operational recommendation system (ORS). Lastly, this section defines both functional and non-functional requirements.

2.1 Elicitation

Requirements elicitation is the structured process through which the project team identifies, collects, and clarifies the needs, expectations, and constraints that the ORS must satisfy. Its purpose is to ensure that the solution addresses real stakeholder goals and operates effectively within the technical, organizational, and regulatory context of the project. Because requirements are often incomplete, ambiguous, or distributed across different sources, elicitation involves not only gathering information but also validating assumptions, resolving inconsistencies, and aligning terminology among stakeholders.

In this project, elicitation serves as the foundation for defining both functional and non-functional requirements, as well as establishing clear acceptance criteria. The outcome of this activity is a consolidated and traceable set of requirements that will guide the subsequent phases of analysis, design, and implementation, reducing the risk of misinterpretation and minimizing costly changes later in the development lifecycle.

As part of the requirements elicitation, the team carried out several interviews with the client, *SNET Energy UK*, as well as with the group project supervisor, Dr. Muge Sayit. The approach during the interviews was to use open questions to explore needs and context and closed questions to confirm details and define specific requirements [3]:

- **Open question – asked to *SNET Energy UK*:** What is the goal of the model?

Answer: *The goal of the model is to generate forecasts and decision-support outputs that maximize profit for the end user. Rather than focusing solely on predictive accuracy, the model is intended to optimize actions that enable users to make optimal operational and market decisions that translate into financial returns by interacting with the electrical grid.*

- **Closed question – asked to *SNET Energy UK*:** Is the goal of the model to make predictions and return them or to return suggested action?

Answer: *The model is designed to return suggested actionable control decisions that optimize profitability based on forecasts (prices, demand, generation, weather, among others). The predictions are intermediate inputs in the model rather than final outputs.*

- **Open question - asked to SNET Energy UK:** How does the Costa Rican market work?

Answer: *The Costa Rican market provides an interesting starting point for creating this type of model because of its centralized structure and simplicity as it is highly regulation-driven with fixed pricing based on prior standards rather than competitive market dynamics.*

- **Closed question – asked to SNET Energy UK:** Is there any room for improving the optimal control choice using optimization modelling in the Costa Rican market, given its simple structure?

- **Answer:** *No. Due to the regulatory and rule-based nature of the Costa Rican market there is limited scope for achieving meaningful performance and profit within the electric market. As a result, the Costa Rican market will be discontinued as a focus, in favor of the more complex and liberalized United Kingdom market, where optimization techniques are expected to deliver greater value.*

These interviews helped understand: the sector; the constraints faced by BESS and PV operators; and the problem SNET Energy UK is aiming to solve (see Section 2.6) with a clear understanding of the ORS goal (see Section 1.1). As a result of these discussions, the team was able to pivot early on from developing an optimisation model for the Costa Rican market, to developing an operational control system for the UK market – as the Costa Rican market structure does not allow for significant gains from optimizing operational system control. Further discussions allowed the team to flesh out the main constraints of the project [3], which are set out in the following bullets:

- **Module deadlines.** The team is constrained by the module deadlines for which this project is being developed, and the resulting limitations on time.
- **Resources.** The team's resources are also limited: they only use open-source external data sources, and each of the nine team members has a limited number of hours to dedicate to the project.
- **Technical limitations.** Also, there are additional constraints, such as technical limitations and limitations imposed by the solution domain itself. These are detailed in the non-functional requirements section (see Section 2.7).

Once the team defined the basic scope of the project, they held validation meetings with both stakeholders – *SNET Energy UK* and Dr. Muge Sayit. These meetings allowed the team to clarify doubts, confirm their understanding of the project, and negotiate requirements when expectations differed between stakeholders [3], [4]. This step was vital to reduce ambiguities and avoid misunderstandings.

Finally, the team transformed the collected information into user stories and system stories. These stories helped describe the expected behavior of the ORS in a clear and structured way and supported later analysis and verification through acceptance criteria [3].

2.2 Analysis

After the initial elicitation, the team analyzed the requirements by first separating them into functional and non-functional requirements, as recommended in standard requirements engineering practice [4]. The non-functional requirements have been divided into categories. The requirements have been checked to ensure they describe a clear system behavior and match the intended workflow of sprints.

Next, the team carried out a structured review of the tasks to remove duplicates, rewrite vague statements into precise “shall” requirements, and define measurable acceptance criteria, which would be verifiable at a later stage through tests [5]. When conflicts appeared, the team used validation meetings to negotiate the final agreement. Each meeting’s decisions have been recorded to maintain traceability from the source to the final requirement.

Finally, the team ranked the requirements by priority to keep the scope feasible within time and resource limits. We assigned priority levels so that lower-priority tasks are scheduled for later sprints. This supports incremental refinement as new feedback is obtained during the project [6].

2.3 Specification

Once the requirements had been elicited and analyzed, the team formalized them through a requirements specification process aimed at ensuring clarity, completeness, consistency, and realism:

- **Clarity.** While developing the requirements, the team continuously reviewed the specification's wording to avoid ambiguous terms and stick to a common glossary of terms. Any technical terminology was directly verified with *SNET Energy UK* in weekly meetings, while managerial terminology was validated through discussions with Dr. Muge Sayit [4].
- **Completeness.** The team has ensured the requirements' completeness through validation meetings where they assessed whether each requirement fully addressed user needs. They also reviewed user stories against stakeholder roles and a range of scenarios to confirm adequate coverage [3]. *SNET Energy UK* has confirmed the completeness of the requirements:

"The continued discussions with the team resulted in a much clearer understanding of the project scope, and changed our strategy from pursuing a Costa Rica-based model to a UK-based model" – Alessandro, SNET Energy UK

- **Consistency and realism.** The identified requirements are realistic. The team carefully considered time and resource constraint considerations in the conception of the requirements. Furthermore, weekly reassessment of ranked priorities ensure no tasks are blocked and milestones can be met in time [5].

The purpose of this stage was to define a stable and usable baseline that could guide design and implementation decisions, while remaining suitable for verification through acceptance criteria and later testing activities.

The project's requirements specification follows an Agile approach. Instead of producing a fixed and complete specification at the start, the project maintains a Product Backlog where requirements are expressed as tasks related to user stories and system stories, and are refined continuously during the project [8], [9], [5]. The team have defined a preliminary set of functional requirements, which may change throughout the project, in Section 2.6 of this document. These have, and will continue to, guide the backlog items for each sprint to keep a consistent baseline and avoid misunderstandings [2], [3]. Each backlog item describes a clear goal and expected system behavior.

To ensure that backlog items are clear and verifiable, each task and story includes acceptance criteria, written in a measurable way. These criteria guide validation and test design and help avoid ambiguous requirements, following recognized requirements-writing practices [5]. Regular backlog refinement ensures that tasks remain well-scoped: large items are split, unclear items are clarified, duplicates are removed, and priorities are updated based on stakeholder feedback and project constraints [6].

Finally, the backlog has been ranked to reflect prioritization with a minimum viable product (MVP) mindset. Core features are placed at the top, while optional improvements and enhancements are scheduled for later iterations. This prioritization supports incremental delivery and frequent alignment with stakeholders, which is a key goal of Agile requirements management [6]. An example can be seen in the first sprint (see Figure 1). For this sprint, the team prioritized research on the tools supporting project management and starting the requirements specification instead of implementing a solution without sufficient project understanding and set team workflows.

☐	Actividad	Persona asignada	Informador	Prioridad	Estado	Resolución	Creada	Actualizada	
☐	☒ GRID-21 Create Req. specs draft and define project aim a...	Yasmine Frizlen	Yasmine Frizlen	Medium	FINALIZADA	Listo	24 ene 2026, 16:40	27 ene 2026, 12:25	
☐	☒ GRID-19 Upload relevant documents to Confluence space...	Ana Patricia Gonzalez	Ana Patricia Go...	Medium	FINALIZADA	Listo	23 ene 2026, 14:39	25 ene 2026, 18:18	
☐	☒ GRID-18 Research BESS architecture	Myriam Raafat	Zabed Emdad	Medium	FINALIZADA	Listo	22 ene 2026, 12:44	30 ene 2026, 16:37	
☐	☒ GRID-15 Research solar panels and battery in BESS-context	Raghad Alsaqrn	Zabed Emdad	Medium	FINALIZADA	Listo	22 ene 2026, 12:43	30 ene 2026, 11:43	
☐	☒ GRID-14 Research country-based BESS grid implementati...	Ana Patricia Gonzalez	Zabed Emdad	Medium	FINALIZADA	Listo	22 ene 2026, 12:41	26 ene 2026, 11:17	
☐	☒ GRID-11 Produce "Project Planning" as parts of the Req. s...	Zabed Emdad	Zabed Emdad	Medium	FINALIZADA	Listo	22 ene 2026, 12:32	28 ene 2026, 10:39	
☐	☒ GRID-10 Research story: Investigate techniques on BESS ...	Sin asignar	Zabed Emdad	Medium	TAREAS POR HACER	Sin resolver	22 ene 2026, 12:18	22 ene 2026, 12:49	
☐	☒ GRID-8 Research story: Research BESS technologies	Sin asignar	Zabed Emdad	Medium	FINALIZADA	Listo	21 ene 2026, 22:13	28 ene 2026, 10:39	

Figure 1: Sprint 1 tasks

2.4 System Architecture

Before defining the optimization model, it is essential to understand the physical architecture of the ORS being controlled. The diagram below shows the energy flow between the grid, photovoltaic field, and BESS, clarifying the operational constraints and decision variables that inform the control system.

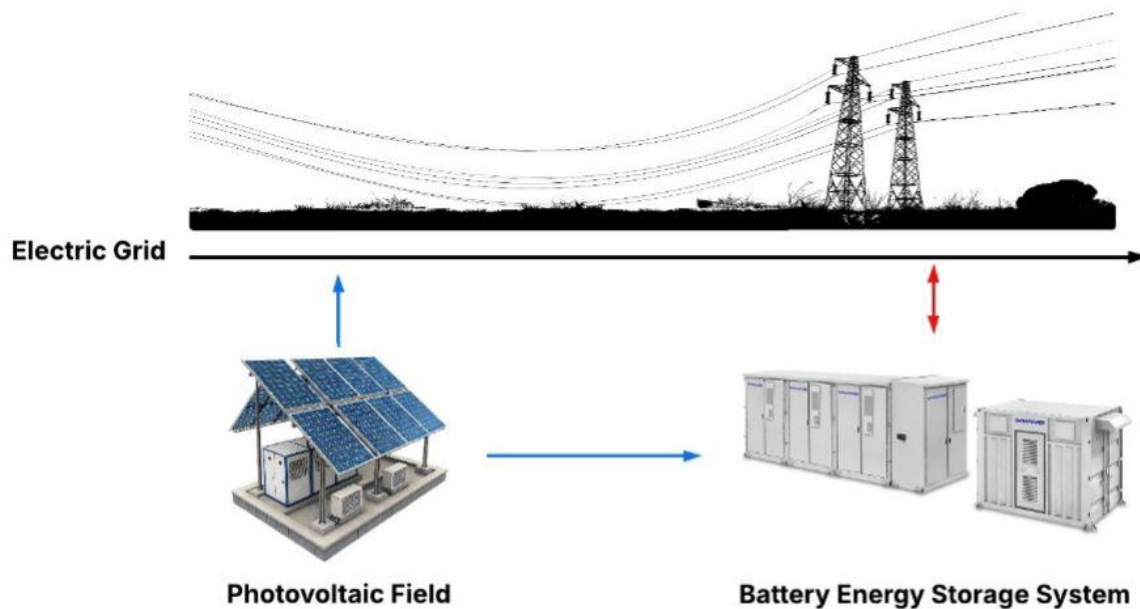


Figure 2: Energy flow of a grid-connected BESS with a photovoltaic field

Each component in this system plays a unique role in the energy generation, storage and distribution process:

- **Electric grid.** The electric grid is responsible for the continuous distribution of electrical energy from power producers to end users and consists of many components such as high-voltage transmission lines, substations, lower voltage distribution networks, among others. This interconnected infrastructure enables the integration of renewable energy (RE) sources, including solar, wind, hydro, geothermal, and biomass generation.
- **Photovoltaic fields.** In the context of this project, photovoltaic fields are considered the primary renewable energy sources. The PV system converts solar energy into electrical energy and is the sole energy generator in this architecture, supplying power either directly to the electric grid or to the BESS for storage. The PV field does not draw energy from either the grid or the battery system.

- **BESS.** The BESS enables bidirectional energy exchange through controlled charging and discharging processes and incorporates battery technologies such as lithium-ion, flow, and lead-acid batteries. The battery system can be charged using surplus photovoltaic generation or by importing energy from the grid during periods of low electricity prices. This allows energy to be stored during periods of excess generation or low demand and discharged back to the grid when required. This flexibility improves system operability and efficiency under varying weather and demand conditions and enables energy storage to participate in electricity markets, where charging and discharging decisions can be leveraged for economic optimization based on time-varying electricity prices.

The final operational recommendation system (ORS) will be designed to maximise profits for the PV battery system in this configuration.

2.5 System Model

With the physical architecture established, the next step is to translate these components and their interactions into a formal system model, which will help define the mathematical relationships, constraints, and decision variables that will need to be considered for the final optimisation model.

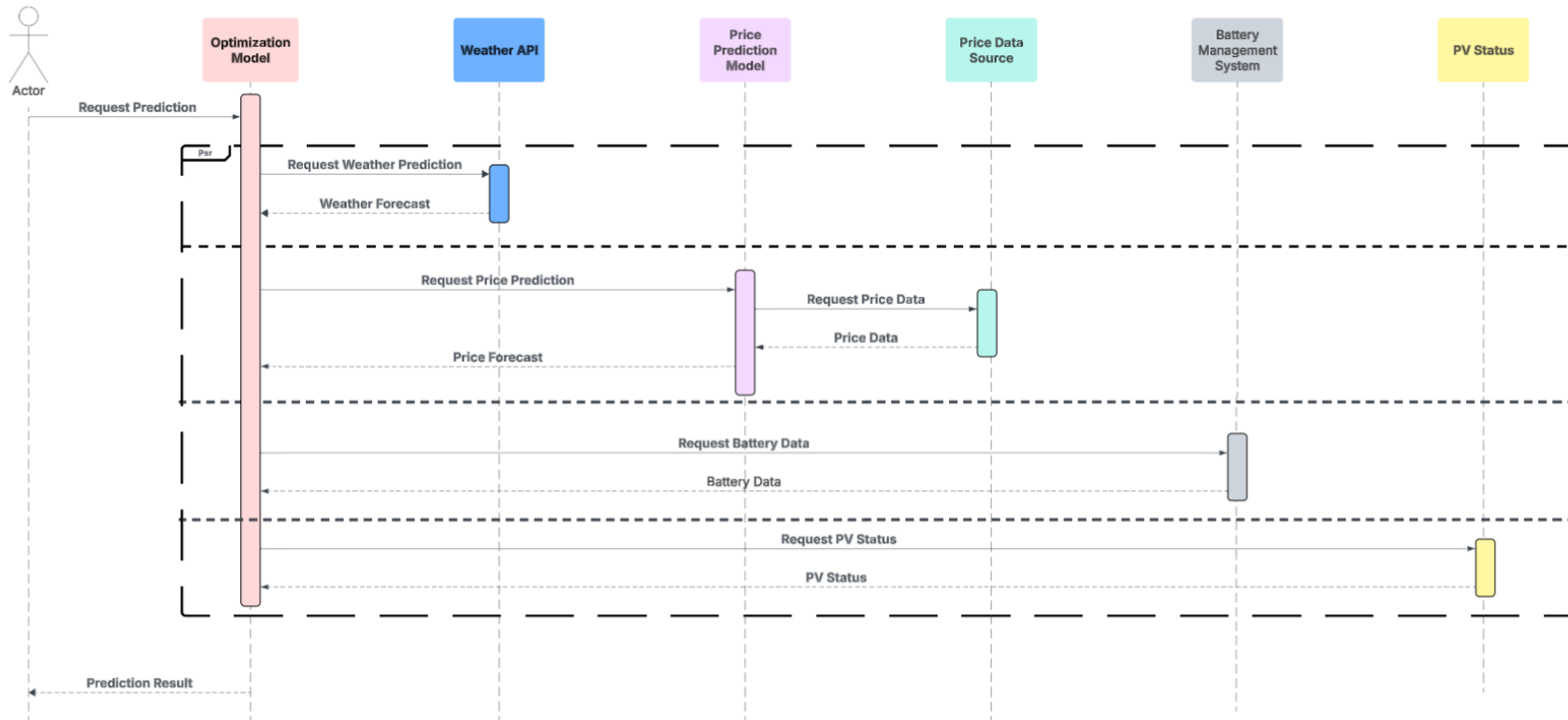


Figure 3: AI-based system for maximum profit in a BES

Figure 3 sets out the ORS model for this project in a sequence diagram. The sequence diagram is a type of interaction diagram that is used to document the most important interactions between objects in a form of sequence. It shows a number of objects and the messages that pass between them within an execution flow [7], [8]. In this project, the objects are made up of the following components:

- **Actor.** The ORS gets triggered by a user request for an action recommendation.
- **Optimization model.** The optimization model gathers all necessary information by triggering the tools at its disposal. This includes a weather prediction API, an energy price prediction model, the state of the Photovoltaic system, and statistics on the efficiency loss and limits of the battery.
- **Weather prediction API.** The weather prediction model retrieves data from an external weather forecast via an API to predict the timing and scale of photovoltaic charging over the next hours.
- **Energy price prediction model.** The energy price prediction model predicts the future energy price levels with a confidence interval for both charge and discharge over the next hours. It will rely on consistent updates on recent energy price levels from the price data source.
- **Price data source.** The price data source will be official charge and discharge price levels, published by Ofgem.
- **Photovoltaic system state.** A key factor in determining the optimal action will be the state of the photovoltaic system. It will pass recent historic data on the generated energy levels on to the optimization model.
- **Battery management system.** The optimization model also retrieves statistics on the efficiency and limitations of the battery, including the state of charge, capacity, charge limit, and discharge limit, and hourly loss, in order to ensure more robust and reliable predictions.

Each component in the ORS model is run in parallel and plays a crucial part in generating a profit-maximizing charging and discharging strategy, at each time interval.

2.6 Functional Requirements

The functional requirement section defines the core services and capabilities the BESS optimization system is expected to provide. They describe how the decisions and computations of the ORS must operate to choose a profit-maximizing recommendation between charging the battery from solar energy generation, charging the battery from the grid, selling energy to the grid from the solar energy generation, and selling energy to the

grid from the discharging battery. Since functional requirements should derive from user stories, Table 2 below presents both the functional requirements as well as their corresponding user stories. Aligned with Agile development practices, this table illustrates how each system requirement is defined into operational perspective.

Functional Requirement	User Story
FR-1: The ORS shall retrieve weather information through an external weather API to predict future solar energy generation. And if possible, obtain weather prediction through the external weather API instead of building our own prediction model.	US-1: As a BESS operations operator, I want the ORS to retrieve relevant weather data from an external weather API, so that the ORS can estimate near-future solar generation to inform operational decisions.
FR-2: The ORS shall use historic and current regional electricity market price data when making operational decisions. The ORS shall generate price predictions along with their associated uncertainty in order to compute an optimal, risk-aware decision.	US-2: As a BESS operations operator, I want the ORS to use current and historical electricity market prices for my selected region, so that recommendations reflect real market conditions and recent price behavior.
FR-3: Within each decision cycle, the ORS shall obtain both weather information and electricity price data before computing an operational decision.	US-3: As a BESS operations operator, I want each decision cycle to obtain both weather/solar information and electricity price data before optimization runs, so that the recommendation is based on a complete and consistent set of inputs.
FR-4: The ORS shall account for energy losses introduced by power conversion and battery storage when estimating the optimal strategy.	US-4: As a BESS operations operator, I want the ORS to account for energy losses from power conversion and battery storage, so that the recommended strategy reflects realistically usable energy and profit.
FR-5: The ORS shall determine the profit-maximizing recommendation based on market prices, solar energy availability, current solar generation, and battery constraints.	US-5: As a BESS operations operator, I want the ORS to compute a profit-maximizing operational recommendation using market prices, solar availability/current generation, and battery constraints, so that I can operate the ORS

	for maximum expected revenue without violating system limits.
FR-6: The outcome of the ORS shall be the profit-maximizing choice between charging the battery from solar energy generation, charging the battery from the grid, selling energy to the grid from the solar energy generation, and selling energy to the grid from the discharging battery.	US-6: As a BESS operations operator, I want the ORS to output one clear profit-maximizing action - charge the battery from solar, charge the battery from the grid, sell solar generation to the grid, or discharge the battery to sell to the grid - so that the decision can be executed unambiguously.

Table 2: Functional Requirement vs User Story

Since the following project follows an Agile development approach, the functional requirements are represented as a product backlog in Figure 4 below using the Jira platform. The backlog illustrates how each functional requirement is captured as an individual work item and how it is prioritized and refined throughout development. This shows the alignment between the specified functional requirement and the Agile planning process. However, each requirement may be refined or extended later during development as the ORS design evolves.

☐	GRID-55 As a BESS operations operator, I want the system to output one clear profit-maximising action—charging from solar or grid, or ...
☐	GRID-54 As a BESS operations operator, I want the system to generate a profit-maximising operational recommendation using market pr...
☐	GRID-53 As a BESS operations operator, I want the system to account for energy losses from power conversion and battery storage, so ...
☐	GRID-52 As a BESS operations operator, I want each decision cycle to obtain both weather/solar information and electricity price data b...
☐	GRID-51 As a BESS operations operator, I want the system to use current and historical electricity market prices for my selected region, ...
☐	GRID-50 As a BESS operations operator, I want the system to retrieve relevant weather data from an external weather API, so that the s...

Figure 4 Backlog representing the BESS optimization

A use case diagram complements the ORS model by mapping out the actors, interactions, and functional requirements that shape how the ORS will be used in practice. Figure 5 describes the interaction between the users and the ORS to provide a description of how a system is used [7].

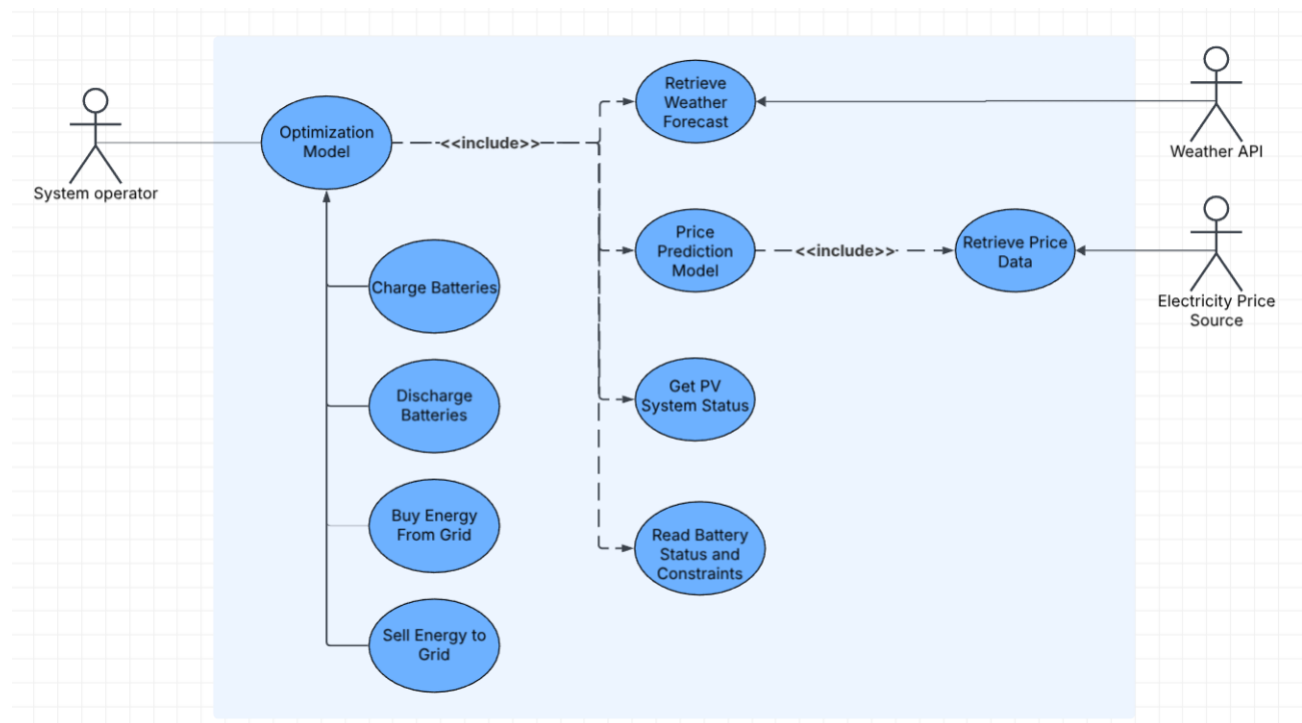


Figure 5 Use case diagram for the project

The diagram explains that the actor, the system operator, initiates the ORS by triggering the optimization model for prediction. The set of prediction recommendations include:

- Buy energy from grid and charge batteries
- Sell energy to grid and discharge battery
- Sell energy to grid from solar plant
- Charge battery with energy from solar plant

In the diagram, these outputs are shown as generalization use cases to the optimization model where at least one output is guaranteed but it depends on the ORS decision prediction which outputs it will be displayed to the operator. The ORS can either recommend the charge or discharge of the battery. Separately, it can also recommend buying from the grid or selling to it.

To perform this prediction, the optimization model requires several important data inputs, which are included as use cases in the diagram. The ORS needs to retrieve weather forecast from a weather API, retrieve price prediction from a price prediction model which includes retrieving price data from an API.

In addition, the ORS obtains real-time data about the physical BESS system. The PV system provides the status of the PV array and the BMS provides information about the batteries including state of charge, capacity limits, and constraints. With these inputs going into the optimization model, the model will be able to predict the optimal decision and provide that prediction to the ORS operator.

2.7 Non-functional requirements

Non-functional requirements constitute a broad category encompassing both those requirements intrinsic to the problem itself and those that extend beyond the ORS direct functionality. Given the breadth of this scope, the team decided to organize them into categories to facilitate understanding.

Product requirements

This section describes requirements that relate to qualities the product or system must possess. These do not focus on features, but rather on aspects such as performance under critical conditions or time constraints, particularly relevant in AI-based systems, for example.

Efficiency requirements

Efficiency requirements define the level of efficiency the ORS must achieve in order to fulfil its intended functions. Among the most common are performance-related requirements, as performance is often a critical factor in software development. This project includes two performance requirements:

- **NFR-PERF-01:** The system shall be able to give a prediction within a reasonable period, after new inputs have been supplied.
- **NFR-PERF-02:** The system shall be able to recompute the predictions and recommendations at provided time intervals (i.e. predictions for each hour, day, etc.).

Reliability requirements

Reliability requirements describe how the ORS is expected to behave in unforeseen situations, such as the absence of data or the failure of an external source.

- **NFR-REL-01:** The system shall be able to recommend prediction if an external data source fails by also alerting the user.
- **NFR-REL-02:** The system shall ensure that the recommendations do not go against the BESS constraints.

- **NFR-REL-03:** The system shall validate ranges and units (e.g., SoC within bounds, non-negative PV generation, correct timestamps) and reject or flag anomalous values.
- **NFR-REL-04:** The system shall define explicit behavior for missing data (imputation, fallback, or “no recommendation”).

Portability requirements

Portability requirements aim to ensure that the solution is replicable and can be executed on different devices without issues, avoiding dependence on a specific operating system, for instance.

- **NFR-PORT-01:** The system shall be deployable on a standard Linux environment using reproducible packaging (e.g., containers) so that the same build runs consistently across environments.

Organizational requirements

There are also requirements driven by the organization itself, such as adherence to good practices, required deliverables, and documentation standards.

Delivery requirements

Delivery requirements specify which documentation must be delivered and in what format.

- **NFR-DEL-02:** The system shall provide deployment documentation listing dependencies and steps to reproduce the runtime environment.

Implementation requirements

Implementation requirements define constraints on how the ORS must be implemented. This often includes modular design practices or maintaining distinct versions.

- **NFR-IMPL-01:** The system shall separate data ingestion, forecasting, recommendation/optimization, and presentation (API/UI) into independent modules to allow changes with minimal impact.
- **NFR-IMPL-02:** The system shall version models and configuration support rollback and reproducibility.

Standards requirements

Standards requirements, as the name suggests, establish minimum criteria that the ORS must meet.

- **NFR-STAN-01:** The system shall meet a minimum predictive performance threshold on a validation set, and it shall report this metric per model version.

External requirements

External requirements arise from the solution domain itself, such as legislation, ethical considerations, privacy, and security. Unlike other requirements are static and cannot be changed throughout the project.

Ethical requirements

Ethical requirements relate to responsible conduct and civic behavior, ensuring respect for others' rights and avoiding unethical actions.

- **NFR-ETH-01:** The system shall use external sources only if they are open source.

Legislative requirements

Legislative requirements are dictated by specific laws and regulations that must be complied with. They are typically critical, as non-compliance may lead to significant harm or legal consequences:

Privacy requirements

- **NFR-PRIV-01:** The system shall not process or use private data unless the data subject has provided explicit consent.
- **NFR-PRIV-02:** The system shall retain evidence for each recommendation (inputs, timestamp, model version, output) for audit for a defined period.

Safety requirements

- **NFR-SAF-01:** Under high uncertainty or inconsistent inputs, the system shall default to a safe action (e.g., “hold/no trade”) and report the reason.

3 Verification, Validation and Testing

To guarantee model functionality and align with stakeholder requirements, the project adopts a comprehensive VV&T approach [1]. Since Agile methodology is adopted, testing and validation will be executed iteratively. Importantly, the final validation test will be completed near the project's final integration. This ensures that the system fulfills the functional requirements and non-functional requirements.

3.1 Verification

Verification focuses on the correctness of the development process and the code produced. Using the Agile approach and adhering to it, progress will be verified against user histories and tasks. Code reviews and CI/CD pipelines will be automated to reduce manual overhead.

3.2 Validation

Validation focuses on the utility of the product in the intended environment; it ensures the solution meets the functional requirements of an ORS; this correlates heavily with the acceptance criteria defined by *SNET Energy UK*.

3.3 Testing

To provide comprehensive analysis of the codebase the ORS will be tested using both inspection-based and execution-based approaches evaluating its functionality and the requirements stated by *SNET Energy UK*.

3.3.1 Testing Approach

Within the ORS different types of testing approaches will be implemented to simplify early defect detection and real execution behavior.

Static Testing

Static testing is performed by inspecting documentation and the codebase via peer-to-peer code reviews and detecting syntax errors or formatting issues.

Dynamic Testing

Dynamic tests observe runtime behaviors of functionalities within the ORS (API calls, training and bug detection).

White Box Testing

White box testing is a developer centric perspective; it provides developers with a clear view on how the code reacts to edge cases and real data.

Black Box Testing

Black box testing focuses on expected behaviors: giving focus to stakeholder-based metrics without including internal transactions within the codebase.

In practice we will use a mix of all four perspective types in order to detect issues early, confirm end-to-end behavior and detect growth and progress in new ORS implementations.

3.3.2 Testing Types

Seven primary test types will be employed to validate the performance of the ORS [1]:

Unit Testing

Unit testing is used to verify individual functions and modules behave correctly in isolation, by evaluating the correctness of specific code logic to ascertain its intended purpose. This includes testing any code-level external dependencies (e.g., weather API, price feeds, optimization solvers) by mocking or stubbing them so that tests remain deterministic and performant.

Class Testing

Class Testing focuses on the verification of individual object-oriented classes within the codebase. We will check alongside the system requirements which objects are going to be needed. Validating that the instance of the objects behaves as expected and can be manipulated correctly is part of this test before it can be integrated into a bigger module.

Integration Testing

Integrating tests verifies multiple modules work together correctly, especially when it concerns shared interfaces, data contracts, and error propagation. The purpose of this testing is to ensure that individual parts of the system continue to function to produce the behavior the team wishes to observe at a higher level. Unlike mocking and stubbing which is done for unit testing, integration testing usually involves creating a controlled environment, through simulations or sandboxing.

System Testing

System Testing serves as the final evaluation of the fully integrated ORS against the initial project specifications and user stories. It's intended to be an end-to-end flow. As described in Table 3: Representation of test cases.

Acceptance Testing

Acceptance testing will be conducted with *SNET Energy UK* to validate if the model works as they expect if the data is available. This phase will be conducted with real-time data to test the model under normal conditions. The objective of this test is to demonstrate the ORS capability to adapt to new data.

Regression Testing:

If a significant restructuring of code or model changes are introduced, regression testing will be done to ensure that new bugs have not been introduced into previously working components. This is important for the project, as different team members are developing different parts of the system simultaneously. Executing this type of testing ensures that integrating new code from different developers across different parts of the system does not break existing functionality.

3.4 System Testing Schedule:

To conclude, the ORS will pass through a System Testing schedule. This approach allows the team to visualize user stories into smaller, manageable functions. The following schedule will be conducted in conjunction with its corresponding implementation (i.e., when a new module is added to the codebase, the corresponding test cases will also be implemented). This cadence of implementation and testing is imperative to successful incremental and iterative deliveries as a key component of Agile-centric development.

Table 3 outlines the testing criteria and cases that the team intends to satisfy during development. This plan is dynamic because of the agile methodology; additional tests, such as class tests, will be defined and incorporated in future sprints as the codebase grows.

Requirements trials							
No.	Test case /Test script	Description of Task	Input	Expected Output	Testing Type	Testing Schedule	Actual Output
1	Extracting weather forecast	Retrieve meteorological data to predict PV generation	API request to external weather service	Hourly solar weather information	Unit Testing	Same sprint as implementation to ensure incremental deliverables meet acceptance criteria.	<i>To be tested</i>
2	Extracting Price Data	Gather current and historical market price levels	API request for official UK price levels	Real time and historic charge/discharge price data	Unit Testing	Same sprint as implementation .	<i>To be tested</i>
3	Parameterization of Model	Configure system limits, efficiency, capacity and place of	Battery capacity, charge/discharge limits,	A configured model ready for training.	Class Testing	Same sprint as implementation .	<i>To be tested</i>

		weather forecast	hourly loss rates				
4	Extracting Battery Levels	Retrieving real-time telemetry from the battery hardware	Battery sensors	Current charge status and available capacity	Unit Testing	Same sprint as implementation .	<i>To be tested</i>
5	Automated feature transformations	Process raw weather and price data into model-ready formats	Raw API data	Normalized data for model input	Integration Testing	Same sprint as implementation .	<i>To be tested</i>
6	Train Price Prediction Model	Update the price model based on recent market shifts	Historical price trends, weather forecast	An updated weights file for price prediction	Unit Testing	Same sprint as implementation .	<i>To be tested</i>
7	Train Optimization Model	Refine the strategy to maximize profit	Historical performance data, efficiency loss stats and transformed features	Optimized logic for buy/sell/charge/dischARGE	Unit Testing	Same sprint as implementation .	<i>To be tested</i>
8	Automated Model Re-Training	Trigger an update for the models where one can assess if the model is deviating	Real-time performance metrics, ground truth data and drift	A drift report confirming model variability. Signal where a	Unit Testing, Integration Testing and Regression Testing	Same sprint as implementation .	<i>To be tested</i>

		itself and needs retraining	thresholds	model retraining is needed			
9	Automated Unit Test Suit Run	Automatically execute all unit tests within the codebase to verify that new additions to the code don't corrupt previous builds.	Latest build from the codebase, with fixed parameters and variables.	All unit tests passed with their own report.	Regression Testing	Every Sprint, including final run prior to deliverable.	<i>To be tested</i>
10	Deployment with real data	Receive real data, a problem configuration and deploy it	Problem parameters, location, connection to batteries	A functional and trainable model with the need of few or no modification to the codebase	Acceptance Testing	Same sprint as implementation	<i>To be tested</i>
11	Validate API Response End to End	Generate internal and external API calls to confirm schema, sanity checks, latency.	API health URLs.	Expected response code 200	Integration Test	Same sprint as implementation	<i>To be tested</i>

12	Internal ORS Performance Metric Validation	Compare ORS performance against an old model to confirm an increase in profit ranging between 15 to 20% or higher.	Baseline performance (old system), Baseline performance (ORS). Profit calculation	ORS reaches $\geq$ 15% profit margin increase.	Acceptance Testing	Same sprint as implementation	To be tested
13	External ORS Performance Metric Validation	Run a final system review with SNET Energy UK with their own acceptance criteria to ensure the ORS meets their requirements.	SNET Energy UK acceptance criteria checklist, sample data, final system report and ORS system	ORS deemed within acceptable criteria for formal signoff	Acceptance Testing	Same sprint as implementation	To be tested

Table 3: Representation of test cases

Agile methodology often requires developers to follow a CI/CD pipeline to sustain iterative cycles of development, testing, and deployment. Maintaining such a workflow manually tends to be time-consuming and error prone. GitLab CI/CD provides a platform that supports automated workflows to enable developers to: execute a pre-defined CI/CD processes; run test suites; and generating pull requests or issues depending on the results generated by the tests [9], [10], [11] [12].

Automation of the ORS test suite will allow asynchronous testing, providing a full run of test prior to generating a .md file with testing outputs as viewable graphs for ease of

understanding. GitLab CI/CD will automatically generate a pull request or issue provided the rules stipulated Table 4 and Table 5.

Field	Example value
Title	CI: <commit-summary> (<branch> → <main>)
Message	Modified files list, test suit description with documentation files and reproduce command.
Rule	Failed test must be 0

Table 4: Boilerplate general format for merge request generation

Field	Example value
Title	CI FAIL: <branch> @ [<file_name>, ...]
Message	Modified files list, test suit description with error codes and reproduce command.
Priority	P0: 7+ failed, P1: 5 failed, P2: 2 failed, P3: 1 failed

Table 5: Boilerplate general format for issue generation

Once each task is finished, it will move into the review state, GitLab CI/CD will run a pipeline on a merge request where an automated testing script will run with the testing and validation dataset seen in Table 3 to run integration, performance, and unit testing. If it passes the tests, a developer will submit a pull request. Successfully running the testing suite; documentation and pull request and proper comments will be needed to approve the request. Else, an issue will be submitted describing where the test failed as seen in Table 4.

During the review state, a reviewer not involved directly with the implementation will enter the pull request, observe documentation and review code formatting to approve it. If there is any reason not to approve, an issue must be submitted with the appropriate format as seen in Table 5.

4 Project Planning

The project planning section serves to define the management approach used to deliver the operational recommendation system. The team adopted an iterative Agile development approach to cater to the uncertainty in data availability and to accommodate the evolving understanding of operational constraints along with the experimental nature of AI and optimization work.

This section will also explore: the tools used to plan, track, and document work; the project plan and timeline; and the progress achieved in initial sprints.

4.1 Project management structure and methodology

Frequent coordination with *SNET Energy UK* stakeholders, the uncertainty in data availability, evolving constraint discovery, and variability in model performance makes an iterative, Agile approach appropriate for this project. Agile describes a set of values and principles focused on shared purposes and vision, scalable network of teams, and the ability to incorporate rapid changes in technology [13]. An agile approach produces incremental delivery with a consistent cadence of regular review points, empowering the team to adapt and change plans as new information is discovered by balancing the overhead proportionate to the scope and timescale of the work [14].

In practice, development teams often practice Agile through frameworks such as Scrum or Kanban. Because the team has adopted Scrumban, this report summarises Scrum and Kanban separately prior to describing how their hybrid approach is ultimately implemented.

Scrum

Scrum is an agile framework for developing complex products by delivering reviewable iterative, incremental work [15]. This is done by practicing incremental delivery in time-boxed periods, typically lasting one working week, known as sprints.

A Scrum team does not have a conventional leader, as leadership is distributed among the roles. The Scrum Master leads the relevant ceremonies and processes, while the Product Owner/Project Manager provides and keeps the team aligned on the vision and priorities of the project, whereas the rest of the team is self-organizing and collectively responsible for delivery [15], [16].

Furthermore, a Scrum environment typically practices some of the major ceremonies involved in such an approach. These are structured meetings such as; sprint planning,

daily standups, sprint review, and sprint retrospective, that foster alignment and transparency. They aid teams plan and track work, gather feedback, and continuously improve processes by assessing and identifying issues early [15], [17].

Additionally, Scrum artifacts such as the sprint backlog, and increment, provide structure and transparency for Agile teams. The backlog lists all desired and potentially upcoming work, and the increment is the sum of the completed work [15], [11]. It refers to the key information needed during the development of a project.

Kanban

Kanban is another agile framework that aids in visualizing work, limiting work-in-progress tasks, and promoting continuous improvement through transparency [18], [19], [20]. A kanban board is typically made up of columns that describe the phases of each task.

A core mechanism is the use of WIP limits to avoid overloading the team with work and to reveal where in the process bottlenecks occur, and by pulling in new work only when capacity is available. In contrast to Scrum's fixed sprint commitments, Kanban supports continuous delivery and is suited to adapt better when priorities shift and tasks arrive unpredictably.

Adopted Methodology: Scrumban

Given the uncertainty typical of research-led AI development, the team adopted a Scrumban approach. Scrumban offers flexibility and continuous delivery by supporting simultaneous initiatives, evolving requirements, and efficient resource allocation [21]. Scrum's events support transparency and regular inspection/adaptation across the sprint, while Kanban practices reduce multitasking and improve throughput by limiting work in progress and managing flow.

From **Scrum**, the team keeps three core practices that provide structure and regular feedback:

- **Sprints.** The team continues to plan and execute work within a time-boxed period.
- **Daily standups.** A daily async standup is held so the team can identify blockers and track progress continuously.
- **Review and Retrospectives.** At the end of every sprint, the team will conduct these two ceremonies in combination, to identify the parts of the process that went well and the parts of the process that need to be improved.

From Kanban, the team adopted three execution controls to manage flow during the sprint:

- **The board.** A kanban board that denotes the phase of each task: a “to do” column, an “in progress” column, a “review” column, a “testing/validation” column, and a “done” column.
- **The tasks/cards.** Each task refined and planned during the Sprint Planning is denoted by a singular and completable task/card.
- **Work-in-progress limits.** For the team to work efficiently, it is imperative to identify the capacity at the beginning of each sprint by estimating the effort of each task and only accepting an appropriate number of tasks for the given sprint.

Scrumban implementation

Sprint cadence and ceremonies

The project operates in short iterations running Thursday to Wednesday (a 7-day sprint cycle). Each sprint begins with Sprint Planning on Thursday, followed by daily asynchronous standups (documented in Confluence), and concludes with a combined Sprint Review and Sprint Retrospective on Wednesday. This cadence preserves Scrum's ceremony structure while considering the individual academic responsibilities each group

member has to their other modules; allowing team members to work over the weekend at their preferred pace.

- **Sprint Planning (Thursday).** Define the sprint goal, confirm capacity, and select work for the sprint backlog.
- **Daily async standup (daily).** A short daily checkpoint to surface progress and blockers, recorded for traceability.
- **Sprint Review + Retrospective (Wednesday).** Review completed work over the duration of the sprint against the goal to identify actionable improvements to apply in the next sprint for continuous improvement.

Roles and responsibilities

The team maintains a consistent Product Owner role (the group leader) to ensure continuity of prioritization and sprint goal clarity. The Scrum Master role rotates between team members to distribute ownership of the process and develop shared competence in running ceremonies and removing impediments.

Effort estimation and task sizing (Planning Poker)

During Sprint Planning, the team uses Planning Poker to estimate effort for each ticket before committing it into the sprint. Planning Poker is a relative estimation technique commonly used by agile teams to align on a shared understanding of complexity and effort.

In this project, estimates use a day-based sizing scale: 2, 3, 4, 5, 5+ days. Items estimated at 5+ are treated as a risk indicator and are typically split into smaller tickets (or deferred) so that sprint commitments remain realistic and the team can maintain a predictable delivery cadence. This aligns with the general agile practice of decomposing large work items to reduce uncertainty and improve flow.

Workflow management: board policies, WIP limits, and “finish-first” prioritization

Execution during the sprint follows Kanban-style flow management using a visual board and explicit working policies. The team limits simultaneous work by applying a WIP constraint that reflects sprint capacity and the number of committed sprint tickets. Controlling WIP is a central Kanban mechanism: it reduces context switching, surfaces bottlenecks, and supports a pull system where work is started only when capacity is available.

To maximize completion and reduce incomplete work, the team adopts a “finish-first” policy: items closer to completion (i.e., nearer the “Done” column) are prioritized over starting new items. This reduces partially completed work and helps the team finish committed items before starting new ones.

Collaboration practices and dependency management

To improve throughput and reduce blocking risk, the team encourages pair or trio working on complex tasks where parallelization is beneficial (e.g., one member implementing while another validates results or documents). In addition, tickets are written to minimize single-ticket dependency chains (i.e., avoiding creating items that are inherently blocked by another ticket wherever possible). This reduces coordination overhead and aligns with the flow objective of keeping work moving through the ORS with minimal waiting states.

Sprint review and retrospective outcomes

Each Wednesday, the team conducts a combined Sprint Review and Retrospective. The Sprint Review inspects completed work and evaluates whether the sprint goal was met; the Retrospective identifies process improvements and commits concrete actions to improve effectiveness in the next sprint. This is treated as a deliberate mechanism to ensure that each sprint incorporates learning and avoids repeating the same failures.

4.2 Project tools

To support the ceremonies the team performs, a small set of project management tools aligned to the overall Scrumban workflow is adopted. These tools provide a single source of truth for work items and sprint progress, a persistent record of decisions and updates, and a collaborative workspace for efficient knowledge sharing.

Jira

Jira is used as the work-tracking system where the team creates, assigns, and monitors project tasks. There are two primary ways the team uses Jira that directly translate to the Scrumban artifacts being used:

- **The Backlog.** During sprint planning, the team creates and refines tasks that need to be done to achieve the goal of that sprint. These tasks exist in the backlog until enough capacity is determined available for the team to tackle them in any given sprint. Figure 6 illustrates the team's backlog at a given point during a sprint.



Figure 6: Backlog of tasks in Jira

- **The Board.** The team also maintains a Kanban board every sprint where the progress of each task is tracked through all relevant phases. The team's Kanban board in sprint 2 is depicted in Figure 7.

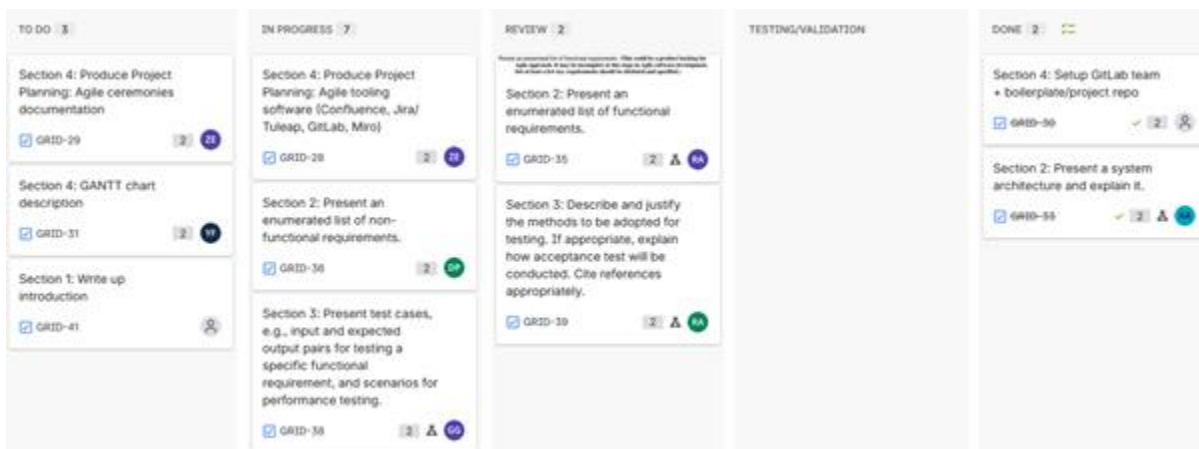


Figure 7: Kanban Board in Jira

Confluence

Confluence is used as the central documentation repository for the project. It aids in providing traceability between the required documentation and deliverables relevant to the solution and provides a space for the team to share information with *SNET Energy UK*. It is also used for daily asynchronous standups, where each team member posts brief progress updates and blockers, to create a persistent audit train of sprint activity.

- **Dashboard.** The figures below display important sections of the team's Confluence dashboard, allowing for easy access to important documents and processes.

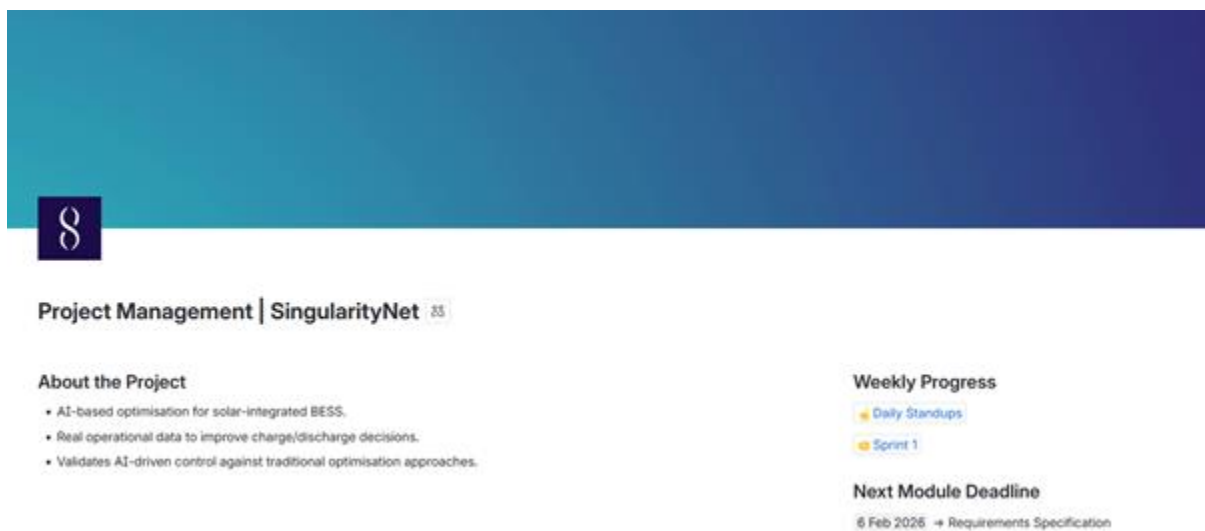


Figure 8: Confluence Dashboard Homepage

Meetings

Board **All** Gallery View +

Table layout Filter Sort 1 Hide field

	Meeting Name	Date	Docs	Status	Notes
1	Sprint 3 Planning	5 Feb 2026		Backlog	
2	Sprint 2 Review	4 Feb 2026		Backlog	
3	Weekly Meeting	2 Feb 2026		Backlog	Meeting focused on reviewing project scope (requirement document! takes priority) and seeking review with Alessandro
4	Meeting with Supervisor Muge	29 Jan 2026		Upcoming	Meeting with supervisor!
5	Clarification Meeting with Alessandro (SingularityNet)	28 Jan 2026		Done	Yasmine had this meeting with Alessandro to clarify the focus of the project prior to Requirement Specifications document is further developed

+ Add entry

Figure 9: Confluence page displaying team meetings at a glance

Jira Tasks

Type	Key	Summary	Assignee	Priority	Status	Updated
	GRID-58	Review of Section 4: Produce Project Planning: Agile tooling software (Conflu...	Del Campo Olano, Pablo	Medium	TO DO	31 Jan 2026, 20:37
	GRID-57	FR8- Use historical and regional electricity market data for decision support		Medium	TO DO	31 Jan 2026, 11:11
	GRID-56	FR7- Battery charge level determination under capacity constraints		Medium	TO DO	31 Jan 2026, 11:09
	GRID-55	As a BESS operations operator, I want the system to output one clear profit-m...		Medium	TO DO	31 Jan 2026, 20:29
	GRID-54	As a BESS operations operator, I want the system to generate a profit-maximi...		Medium	TO DO	31 Jan 2026, 20:27
	GRID-53	As a BESS operations operator, I want the system to account for energy losse...		Medium	TO DO	31 Jan 2026, 20:25
	GRID-52	As a BESS operations operator, I want each decision cycle to obtain both wea...		Medium	TO DO	31 Jan 2026, 20:25
	GRID-51	As a BESS operations operator, I want the system to use current and historical...		Medium	TO DO	31 Jan 2026, 20:25
	GRID-50	As a BESS operations operator, I want the system to retrieve relevant weather ...		Medium	TO DO	31 Jan 2026, 20:05
	GRID-49	Review 2	Zabeed Emrad	Medium	done	31 Jan 2026, 09:43
	GRID-48	Review	Yasmine Frizlen	Medium	done	30 Jan 2026, 18:46

Figure 10: Confluence page displaying Jira tasks at a glance

- **Document Hub.** The following figures depict pages in Confluence where the team's internal documentation, and external documentation from *SNET Energy UK* are maintained, respectively.

Internal Documents

All Board					
Table layout Filter Sort Hide field					
☐	Name	Tag field	Link	Jira Task	Media and file...
1	Requirements Specification	Module Deadline			
2	Final Programming Code and Report	Module Deadline			
3	Presentation	Module Deadline			
4	Country Based Research	Research	https://essexuniversity-my.sharepoint.co...	GRID-14: Research country-based BESS grid ...	
5	System Requirements Specification and Project Plan	Deliverable	https://essexuniversity-my.sharepoint.co...	GRID-21: Create Req. specs draft and define ...	
6	Research BESS architecture	Research	https://essexuniversity-my.sharepoint.co...	GRID-16: Research BESS architecture done	
7	Research solar panels and	Research	https://essexuniversity-my.sharepoint.co...	GRID-15: Research solar panels and battery l...	

Figure 11: Confluence page where internal documents are maintained

Table layout Filter Sort Hide field			
☐	Text field	Tag field	Link
1	BESS mockup and physics constraints	Research Created by Ale...	https://essexuniversity-my.sharepoint.com/x/jg/personal/y12551t_essex_ac_uk/1Q8opzXayC4N5L8ZquGK22JAV_awL4mM5OLvgKZe-yu0iKE7e-pjo...
2			
3			

+ Add entry

Figure 12: Confluence page where external documents are maintained

- **Daily Asynchronous Standups.** As part of the team’s commitment to adhering to Scrum ceremonies as part of the chosen methodology, daily asynchronous standups are held in Confluence. As opposed to conducting them in real-time meetings, performing it asynchronously presented the liberty for team members to engage with the project despite their other academic obligations. The figure below depicts an example day of a standup and the general template used.

📅 Sunday 25 Jan 2026

	Name	Priorities 📌	Progress 🏃	Problems 🤔
1	@Del Campo Olano, Pablo	Review on GRID-15: Research solar panels and battery in BESS-context DONE		
2	@Galvan Garcia, Gabriel	Researched methods to automate GitHub workflow in order to automatically test, document and generate pull-request when trying to merge.		
3	@Raghad Alsaqran	wait for document to be reviewed	document was reviewed	
4	@Yasmine Frizlen	• Create GANTT chart • Finished architecture review • Finished project objectives and aim + doc structure		
5	@Zabed Emdad	• Set up Miro for hosting Retro • Complete Project Planning task	• Completed most of the Project Planning ◦ Need to clean up formatting and structure.	• Still awaiting Tuleap access.
6	@Ricardo Kaleb Flores Alfonso	Reviewed Ana Paty work on GRID-14: Research country-based BESS grid implementations DONE	Finished reviewing	
7	@Eduardo Dabbah	Find papers on mathematical methods for BESS optimization		
8	@Myriam Raafat	• Received feedback but i needed more information in order to complete the missing diagram		
9	@Ana Patricia Gonzalez Alcocer	Working on GRID-14: Research country-based BESS grid implementations DONE with @Ricardo Kaleb Flores Alfonso .	Adapted Confluence space, it's ready for review and feedback!	

Figure 13: Asynchronous Standup for Saturday 25-01-2026

Miro

The team uses Miro - a workspace for teams to participate in collaborative design and brainstorming - for whiteboarding during retrospectives, which are held at the end of every sprint. A structured retrospective template with four-quadrant format is used to gather feedback, group themes, and produce actionable improvements.

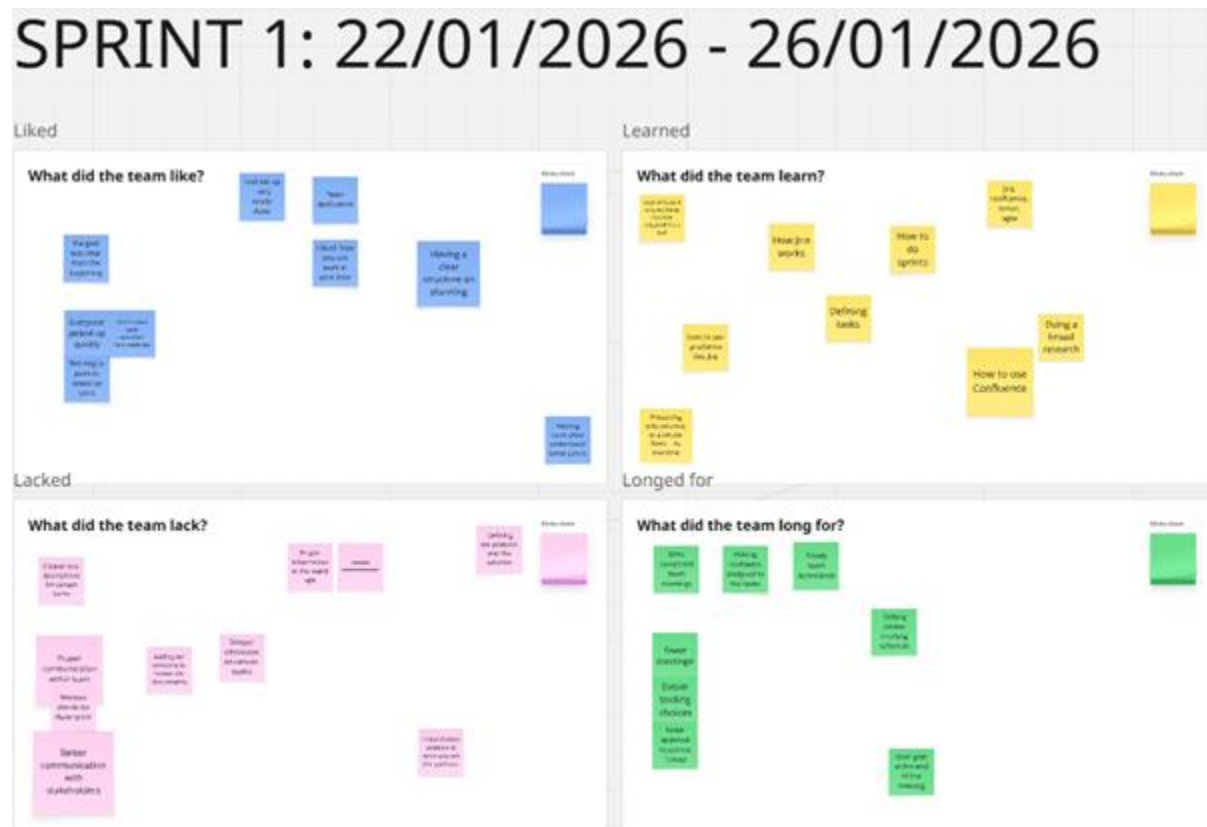


Figure 14: Sprint 1 Retrospective held in Miro

Kollabe

Kollabe is a free online tool that allows teams to participate in Planning Poker. The team has decided to use this service during Sprint Planning to estimate the effort of tasks after having refined them, such that they may be appropriately assigned to a sprint should there be sufficient capacity within the team. Figure 15 illustrates the Kollabe lobby the team uses during sprint planning to conduct point estimation for the tasks of the given sprint.

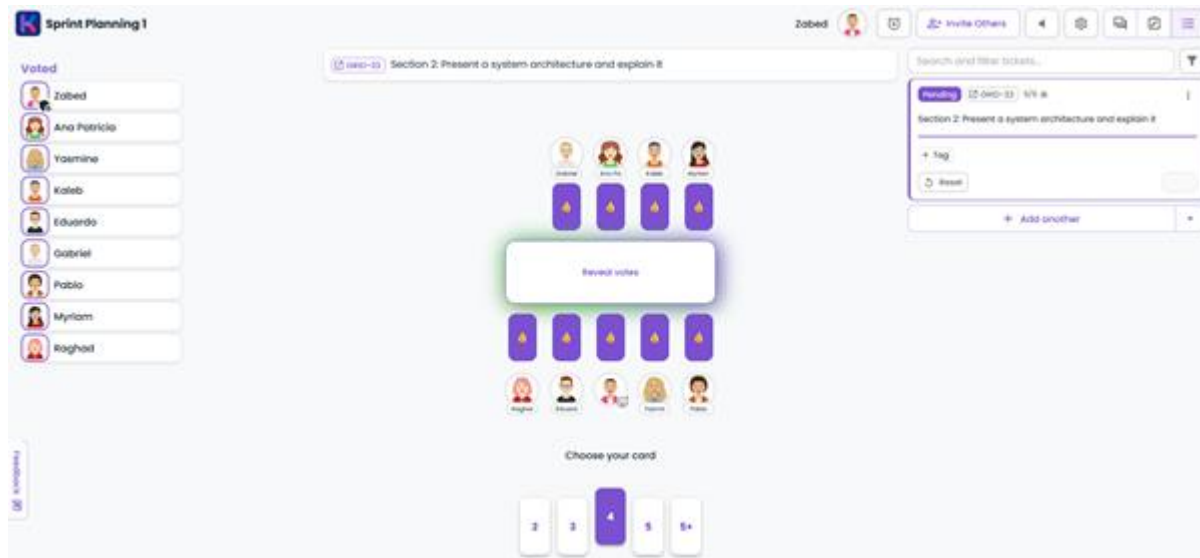


Figure 15: Kollabe lobby where the team conducts Planning Poker

4.3 Project plan

The GANTT chart (see Figure 16) provides a preliminary work breakdown, associated with a time estimation, major milestone identification, and activity sequencing. The project milestones provide the team with intermediate goals and progress reference points. These will function as deliverables in the project's sprints – in line with the Agile approach.

Although the chart also includes slack time, the GANTT chart functions as an indication only since the project follows an Agile approach. Replanning will thus be done in line with sprints, and a continued review process as needed.

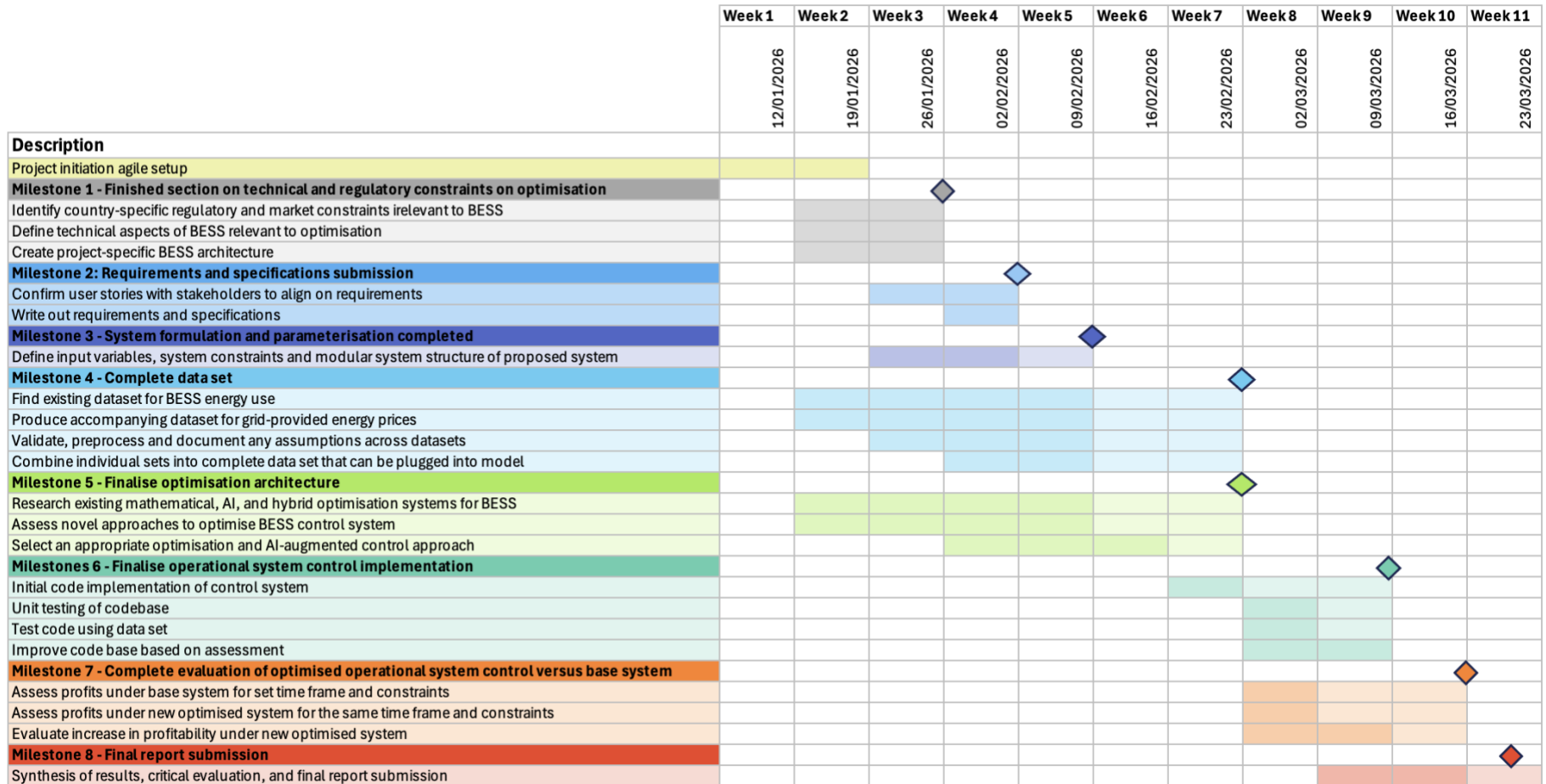


Figure 16: GANTT chart of project plan

Description: Lighter colors in the chart indicate float (or slack) time. Where no lighter shade exists, there is no slack time

4.4 Progress report

In the first sprint, the team decided to focus on: establishing a baseline understanding of how a grid-connected solar + battery energy storage system (BESS) would operate and is ultimately monetized; and set up the team's working environment so that requirements and subsequent implementation can proceed in a structured and traceable way. Table 6 shows the tasks the team decided to tackle during the first sprint. The goal of the first sprint was: **Research preliminary topics for Monday call with SNET Energy UK.**

Type	Work item	Key scope / focus
Research story	BESS research (domain discovery)	- Grids: Costa Rica + UK laws, pricing, schedules, limitations; buy/sell capability - Solar panels: charging the battery via PV generation - Battery: solar OR grid charging (not both); specs, rates, degradation considerations - Architecture: high-level hardware workflow
Task	Research solar panels and battery in BESS-context	- Solar-to-battery interaction and operational constraints - Charge source exclusivity (solar OR grid) - Battery specs/material variation; charge/discharge limits; degradation risk
Task	Research BESS architecture	- BESS infrastructure components - Electricity flow from source to destination - Process diagrams to support understanding
Task	Research country-based BESS grid implementations	- Costa Rica + UK pricing mechanisms and schedules - Limitations (technological/economic) - How buying/selling energy to the grid works
Task	Create requirements specification draft + define aim/objectives (Section 1)	- Tailor formatting to group project - Write project aim and objectives - Add prompts/structure for requirements sections
Task	Upload documents to Confluence + set up team dashboard	- Create and organize the Confluence space - Upload relevant documents - Configure dashboard for visibility
Task	Produce Project Planning content within the requirements specification	- Document planning approach (iterations/sprints) - Describe tracking/documentation practices
Untracked task	Set up Jira for the team (pre-sprint tooling enablement)	- Create Jira project space and workflow configuration - Enable backlog and ticket tracking for later sprints

Table 6: Table of tasks undertaken by the team in the first sprint

References

- [1] C. Dawson, *Projects in Computing and Information Systems : A Student's Guide*, Pearson Education, Limited, 2015.
- [2] G. T. Doran, "There's a S.M.A.R.T. way to write managements's goals and objectives.," *Management Review*, vol. 70, no. 11, pp. 35-36, 1981.
- [3] ISO/IEC/IEEE, "Systems and software engineering—Life cycle processes—Requirements engineering," 2018. [Online]. Available: <https://www.iso.org/obp/ui/en/#iso:std:iso-iec-ieee:29148:ed-2:v1:en>. [Accessed 28 January 2026].
- [4] University of Essex, Lecture 34: Requirements engineering CE 903 Group Projec.
- [5] INCOSE Requirements Working Group, "Guide to writing Requirements," 2023. [Online]. Available: https://www.incose.org/docs/default-source/working-groups/requirements-wg/gtwr/incose_rwg_gtwr_v4_040423_final_drafts.pdf?sfvrsn=5c877fc7_2. [Accessed 28 January 2026].
- [6] K. S. a. J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum," 2020.
- [7] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Boston, MA, USA, 2003.
- [8] N. S. a. D. Vega, *Fundamentals of Software Engineering*, Sebastopol, CA, USA, 2025.
- [9] GitLab, "CI/CD Pipelines".
- [10] GitLab, "Issues API".
- [11] C. Harris, "Agile scrum artifacts," n.d.. [Online]. Available: <https://www.atlassian.com/agile/scrum/artifacts>. [Accessed 1 January 2026].
- [12] GitLab, "Merge Requests API".

- [13] M. B. A. v. B. A. C. W. C. M. F. J. G. J. H. A. H. R. J. J. K. B. M. R. C. M. S. M. K. S. J. S. D. T. Kent Beck, “Manifesto for Agile Software Development,” 2001. [Online]. Available: <https://agilemanifesto.org>.
- [14] S. Al-Saqqah, Agile Software Development: Methodologies and Trends, International Journal of Interactive Mobile Technologies, 2020.
- [15] J. S. Ken Schwaber, “The Scrum Guide,” 2020.
- [16] D. West, “Agile scrum roles and responsibilities,” n.d.. [Online]. Available: <https://www.atlassian.com/agile/scrum/roles>. [Accessed 26 January 2026].
- [17] D. Radigan, “A guide to agile ceremonies and scrum meetings,” n.d.. [Online]. Available: <https://www.atlassian.com/agile/scrum/ceremonies>. [Accessed 26 January 2026].
- [18] Kanban University, “The Kanban Method,” n.d.. [Online]. Available: <https://kanban.university/kanban-guide/>. [Accessed 26 January 2026].
- [19] D. Radigan, “Kanban,” n.d.. [Online]. Available: <https://www.atlassian.com/agile/kanban>. [Accessed 26 January 2026].
- [20] Kanban University, “Principles and Practices of the Kanban Method,” n.d.. [Online]. Available: <https://kanban.university/kanban-guide/#principles-practices>. [Accessed 26 January 2026].
- [21] Atlassian, “Scrumban: Mastering two Agile methodologies,” n.d.. [Online]. Available: <https://www.atlassian.com/agile/project-management/scrumban>. [Accessed 1 January 2026].